



Universidade Federal de Campina Grande  
Centro de Competência EMBRAPPI VIRTUS em Hardware Inteligente  
para a Indústria

## Guia de Referência

Arquitetura e Operação do Timer EF\_TMR32 (Wishbone)

**Orientadores:** Professor Dr. Marcos Ricardo Alcântara Morais  
Professor Dr. Gutemberg Gonçalves dos Santos Júnior

Campina Grande — PB

15 de julho de 2026

---

# Sumário

|          |                                                           |           |
|----------|-----------------------------------------------------------|-----------|
| <b>1</b> | <b>Introdução Arquitetural</b>                            | <b>1</b>  |
| <b>2</b> | <b>Pinagem e Mapa de Sinais</b>                           | <b>2</b>  |
| 2.1      | Interface de Barramento Wishbone (Slave) . . . . .        | 2         |
| 2.2      | Sinais de Interface Funcional (I/O) . . . . .             | 4         |
| <b>3</b> | <b>Operação do Núcleo</b>                                 | <b>5</b>  |
| 3.1      | A Hierarquia de Clock e o Prescaler . . . . .             | 5         |
| 3.2      | Modos de Contagem e Comportamento de Transição . . . . .  | 5         |
| <b>4</b> | <b>Mapa de Registradores e Funcionalidade de Bits</b>     | <b>7</b>  |
| 4.1      | Tabela de Registradores . . . . .                         | 7         |
| 4.2      | Detalhamento de Bits . . . . .                            | 8         |
| 4.2.1    | Registradores TMR, RELOAD, PR, CMPX, CMPY . . . . .       | 8         |
| 4.2.2    | Registrador CTRL (0x0014) . . . . .                       | 9         |
| 4.2.3    | Registrador CFG (0x0018) . . . . .                        | 10        |
| 4.2.4    | Registradores PWM0CFG (0x1C) e PWM1CFG (0x20) . . . . .   | 10        |
| 4.2.5    | Registrador PWMDT (0x0024) . . . . .                      | 13        |
| 4.2.6    | Registrador PWMFC (0x0028) . . . . .                      | 14        |
| 4.2.7    | Registradores de Interrupção (IM, RIS, MIS, IC) . . . . . | 14        |
| 4.2.8    | Registrador GCLK (0xFF10) . . . . .                       | 15        |
|          | <b>Referências</b>                                        | <b>16</b> |

Tabela 1: Revisão Histórica da Documentação

| <b>Versão</b> | <b>Data</b>         | <b>Autor(es)</b> | <b>Descrição das Alterações</b> |
|---------------|---------------------|------------------|---------------------------------|
| 2.0           | 15 de julho de 2026 | Felipe Fernandes | Documentação de uso do TMR_32.  |

## **Resumo**

Este manual oferece uma análise estrutural e arquitetural detalhada do IP EF\_TMR32. O documento consolida a interface física do módulo, detalhando a pinagem do barramento Wishbone B4 em modo escravo e os sinais de entrada e saída funcionais. Além disso, explora a operação do núcleo de contagem — incluindo a hierarquia de clock e os modos de transição — e apresenta um mapeamento dos registradores de controle, destacando suas funcionalidades em nível de bit. As informações foram validadas a partir do código RTL e do datasheet oficial, fornecendo a base técnica necessária para a compreensão e integração em nível de hardware.

# 1 Introdução Arquitetural

O EF\_TMR32 não é meramente um contador de pulsos; ele é um **processador de sinais de temporização autônomo**. Projetado para desonerar a CPU principal, ele gerencia tarefas complexas de modulação e temporização diretamente no hardware. Sua arquitetura modular e a interface Wishbone padronizada facilitam a integração em sistemas baseados em microcontroladores ou FPGAs.

## 2 Pinagem e Mapa de Sinais

A interface física do módulo é projetada para ser robusta e fácil de rotear. As informações a seguir foram consolidadas do datasheet e do código RTL.

### 2.1 Interface de Barramento Wishbone (Slave)

O barramento segue a especificação Wishbone B4 em modo escravo. Para compreender a integração do EF\_TMR32 em um SoC (System on Chip), é fundamental visualizar a topologia de conexão.

A Figura 2.1 ilustra a interligação padrão ponto a ponto entre um Mestre e um Escravo Wishbone.

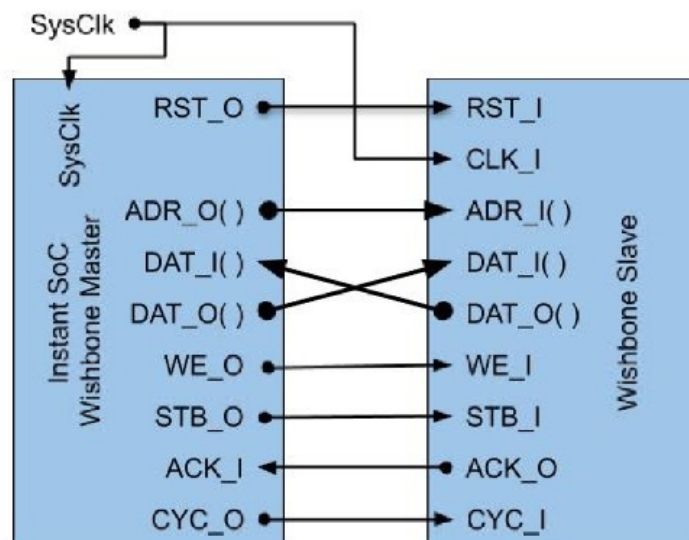


Figura 2.1: Diagrama de conexão padrão entre Mestre e Escravo no barramento Wishbone.

Conforme observado no diagrama:

- **Sincronismo:** Ambos os módulos compartilham o mesmo domínio de clock (**CLK\_i**), garantindo a sincronia das transações.

- **Fluxo de Dados Cruzado:** As vias de dados são unidirecionais. A saída de dados do mestre (DAT\_O) conecta-se à entrada do escravo (DAT\_I), e a saída do escravo conecta-se à entrada do mestre.
- **Handshake:** Os sinais CYC e STB fluem do mestre para o escravo para iniciar a transação, enquanto o sinal ACK flui no sentido inverso para sinalizar a conclusão.

A tabela a seguir detalha a função de cada um desses pinos físicos presentes no módulo EF\_TMR32:

| Sinal        | Direção | Descrição Detalhada                                                                                                                                                                  |
|--------------|---------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| clk_i        | Input   | <b>Clock do Sistema:</b> Todas as operações internas são sincronizadas na borda de subida deste sinal.                                                                               |
| rst_i        | Input   | <b>Reset Síncrono:</b> Quando alto, reinicia todos os registradores para seus valores padrão. Deve ser mantido alto por pelo menos um ciclo de clock.                                |
| adr_i [31:0] | Input   | <b>Barramento de Endereço:</b> Seleciona o registrador alvo. (Não utilizando os 32 bits – há um offset)                                                                              |
| dat_i [31:0] | Input   | <b>Dados de Escrita:</b> Dados enviados pelo mestre para serem armazenados nos registradores.                                                                                        |
| dat_o [31:0] | Output  | <b>Dados de Leitura:</b> Conteúdo do registrador selecionado enviado de volta ao mestre. (Leitura via WB).                                                                           |
| sel_i [3:0]  | Input   | <b>Seleção de Byte:</b> Permite operações em partes da palavra de 32 bits (essencial para compatibilidade com diferentes larguras de dados). [Atualmente, ele não está funcionando]. |
| cyc_i        | Input   | <b>Cycle:</b> Indica que um mestre deseja realizar um ou mais ciclos de barramento.                                                                                                  |
| stb_i        | Input   | <b>Strobe:</b> Indica que o endereço e os sinais de controle atuais são válidos.                                                                                                     |
| we_i         | Input   | <b>Write Enable:</b> Define se a operação é de Escrita (1) ou Leitura (0).                                                                                                           |

| Sinal | Direção | Descrição Detalhada                                                                                          |
|-------|---------|--------------------------------------------------------------------------------------------------------------|
| ack_o | Output  | <b>Acknowledge:</b> Sinaliza ao mestre que a operação foi concluída com sucesso. (Os dados foram recebidos). |

## 2.2 Sinais de Interface Funcional (I/O)

| Sinal       | Direção | Descrição Detalhada                                                                                                                                                                                               |
|-------------|---------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| IRQ         | Output  | <b>Interrupt Request:</b> Sinal de nível alto que indica um evento de interrupção (Timeout, Match X ou Match Y). Permanece alto até ser limpo via registrador IC.                                                 |
| pwm0 / pwm1 | Output  | <b>Saídas PWM:</b> Sinais de onda quadrada gerados pela lógica de comparação. Podem ser invertidos ou atrasados pelo <i>dead time</i> . São desabilitados em caso de falha.                                       |
| pwm_fault   | Input   | <b>Entrada de Falha:</b> Um sinal assíncrono que, quando ativado, força as saídas PWM para o estado inativo instantaneamente via hardware, ignorando o controle de software. É um mecanismo de segurança crítico. |



## 3 Operação do Núcleo

### 3.1 A Hierarquia de Clock e o Prescaler

O fluxo de tempo no IP começa no sinal `clk_i`. Antes de atingir o contador principal, o clock passa por dois estágios:

1. **Clock Gating (GCLK):** Uma porta lógica física controlada pelo bit 0 do registrador GCLK. Quando desabilitado ( $GCLK[0]=0$ ), o clock `clk_g` para o núcleo do timer é cortado, economizando energia. O barramento Wishbone, no entanto, continua operando com `clk_i`, permitindo acesso aos registradores mesmo com o timer inativo [3]. Entretanto, caso o GCLK não seja ativado, há uma condição em que o timer funciona, somente, com o `clk_i`.
2. **Prescaler (PR):** Um contador regressivo de 32 bits (configurável via parâmetro PRW) que atua como um divisor de frequência. Se o registrador PR contiver o valor  $N$ , o contador principal só será incrementado/decrementado a cada  $N + 1$  ciclos de `clk_g`. A frequência efetiva de contagem é  $F_{clk\_g}/(PR + 1)$ . Isso permite que um clock de sistema de 100MHz, por exemplo, controle eventos que ocorrem em intervalos de segundos.

### 3.2 Modos de Contagem e Comportamento de Transição

O comportamento do contador (TMR) nos limites é definido pelo registrador CFG:

- **Modo Up** ( $CFG[1:0]=10$ ): O contador inicia em 0 e sobe até igualar RELOAD. Ao atingir RELOAD, ele gera um sinal de *Timeout*. Se o modo for **Periódico** ( $CFG[2]=1$ ), ele reseta para 0 no ciclo seguinte e reinicia. Se for **One-shot** ( $CFG[2]=0$ ), ele para em RELOAD e desabilita o bit TE (Timer Enable) no registrador CTRL.
- **Modo Down** ( $CFG[1:0]=01$ ): O contador é carregado com o valor de RELOAD e decresce até 0. Ao atingir 0, gera o *Timeout*. O comportamento Periódico/One-shot é análogo ao modo Up.
- **Modo Up-Down** ( $CFG[1:0]=11$ ): Este modo é essencial para PWM de fase correta. O contador sobe de  $0 \rightarrow RELOAD$  e, ao atingir o topo, inverte a direção

automaticamente para  $RELOAD \rightarrow 0$ . Ao atingir 0, inverte novamente para Up. Isso cria uma forma de onda triangular, ideal para minimizar harmônicos em controle de motores e aplicações de áudio.

## 4 Mapa de Registradores e Funcionalidade de Bits

O controle do IP é realizado através de 16 registradores principais, acessíveis via barramento Wishbone. O acesso é de 32 bits, mas a largura efetiva de alguns registradores pode ser menor. Os offsets são baseados no datasheet e verificados com o RTL.

### 4.1 Tabela de Registradores

| Nome    | Offset | Reset | Acesso | Descrição                                                                                   |
|---------|--------|-------|--------|---------------------------------------------------------------------------------------------|
| TMR     | 0x0000 | 0x0   | R      | Valor atual do contador do Timer. Leitura de 32 bits.                                       |
| RELOAD  | 0x0004 | 0x0   | RW     | Valor de carga/limite do contador. Escrita/Leitura de 32 bits.                              |
| PR      | 0x0008 | 0x0   | RW     | Valor do Prescaler (Divisor de frequência). Escrita/Leitura de 32 bits (PRW bits efetivos). |
| CMPX    | 0x000C | 0x0   | RW     | Registrador de Comparação X (para PWM). Escrita/Leitura de 32 bits.                         |
| CMPY    | 0x0010 | 0x0   | RW     | Registrador de Comparação Y (para PWM). Escrita/Leitura de 32 bits.                         |
| CTRL    | 0x0014 | 0x0   | RW     | Controle de operação do Timer e PWM. Escrita/Leitura de 7 bits efetivos.                    |
| CFG     | 0x0018 | 0x0   | RW     | Configuração de modo e direção. Escrita/Leitura de 3 bits efetivos.                         |
| PWM0CFG | 0x001C | 0x0   | RW     | Configuração de eventos para PWM 0. Escrita/Leitura de 12 bits efetivos.                    |
| PWM1CFG | 0x0020 | 0x0   | RW     | Configuração de eventos para PWM 1. Escrita/Leitura de 12 bits efetivos.                    |
| PWMDT   | 0x0024 | 0x0   | RW     | Configuração do Dead Time. Escrita/-Leitura de 8 bits efetivos.                             |

| Nome  | Offset | Reset | Acesso | Descrição                                                       |
|-------|--------|-------|--------|-----------------------------------------------------------------|
| PWMFC | 0x0028 | 0x0   | W      | Limpeza de falha do PWM. Escrita de 16 bits efetivos.           |
| IM    | 0xFF00 | 0x0   | RW     | Máscara de Interrupção. Escrita/Leitura de 3 bits efetivos.     |
| MIS   | 0xFF04 | 0x0   | R      | Status de Interrupção Mascarada. Leitura de 3 bits efetivos.    |
| RIS   | 0xFF08 | 0x0   | R      | Status de Interrupção Bruta (Raw). Leitura de 3 bits efetivos.  |
| IC    | 0xFF0C | 0x0   | W      | Limpeza de Interrupção. Escrita de 3 bits efetivos.             |
| GCLK  | 0xFF10 | 0x0   | RW     | Habilitação de Clock Gating. Escrita/-Leitura de 1 bit efetivo. |

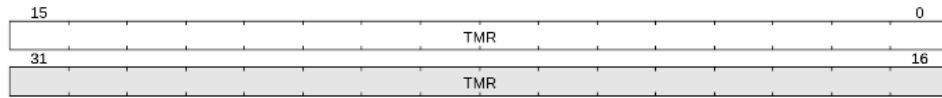
## 4.2 Detalhamento de Bits

### 4.2.1 Registradores TMR, RELOAD, PR, CMPX, CMPY

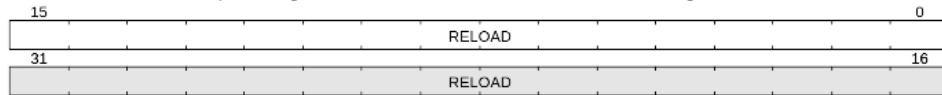
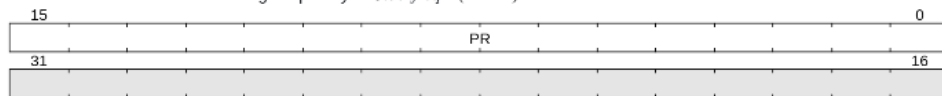
Estes registradores de 32 bits controlam os valores fundamentais de contagem e comparação do timer. Suas estruturas são diretas, com o valor ocupando a totalidade dos 32 bits, conforme ilustrado na Figura 4.1.

**TMR Register [Offset: 0x0, mode: r]**

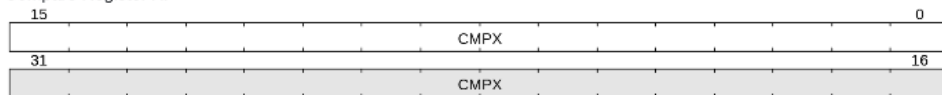
The current value of the Timer.

**RELOAD Register [Offset: 0x4, mode: w]**

The timer reload value. In up counting it is used as the terminal count. For down counting it is used as the initial count.

**PR Register [Offset: 0x8, mode: w]**The Prescaler. The timer counting frequency is  $Clockfreq / (PR + 1)$ **CMPX Register [Offset: 0xc, mode: w]**

Compare Register X.

**CMPY Register [Offset: 0x10, mode: w]**

Compare Register Y.

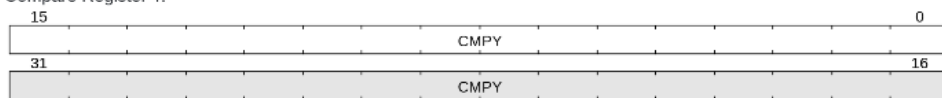


Figura 4.1: Diagramas de Mapeamento de Bits para TMR, RELOAD, PR, CMPX, CMPY e início de CTRL.

### 4.2.2 Registrador CTRL (0x0014)

Este registrador de 7 bits controla as operações básicas do timer e PWM. A Figura 4.2 detalha seu mapeamento de bits.

- **Bit 0 (TE - Timer Enable):** 1: Habilita a contagem do timer. 0: Pausa a contagem, preservando o valor atual do TMR.
- **Bit 1 (TS - Timer Start):** Usado no modo One-shot. Escrever 1 e, em seguida, 0 reinicia o timer. É um pulso de disparo para iniciar uma nova contagem no modo One-shot.
- **Bit 2 (POE - PWM0 Output Enable):** 1: Habilita a saída no pino `pwm0`. 0: Desabilita a saída, forçando `pwm0` a um estado inativo (geralmente baixo).

- **Bit 3 (P1E - PWM1 Output Enable):** 1: Habilita a saída no pino `pwm1`. 0: Desabilita a saída, forçando `pwm1` a um estado inativo.
- **Bit 4 (DTE - Dead Time Enable):** 1: Habilita o gerador de *dead time* para os sinais PWM. 0: Desabilita o *dead time*.
- **Bit 5 (PI0 - PWM0 Invert):** 1: Inverte logicamente o sinal final de `pwm0`. 0: Saída `pwm0` não invertida.
- **Bit 6 (PI1 - PWM1 Invert):** 1: Inverte logicamente o sinal final de `pwm1`. 0: Saída `pwm1` não invertida.

### 4.2.3 Registrador CFG (0x0018)

Este registrador de 3 bits define o modo de contagem e operação do timer. A Figura 4.2 detalha seu mapeamento de bits.

- **Bits 1:0 (DIR - Direction):** Define a direção de contagem do timer.
  - 10 (Up): O contador incrementa de 0 até RELOAD.
  - 01 (Down): O contador decrementa de RELOAD até 0.
  - 11 (Up-Down): O contador alterna entre incrementar (0 a RELOAD) e decrementar (RELOAD a 0).
- **Bit 2 (P - Periodic):** Define o comportamento do timer ao atingir o limite.
  - 1 (Periódico): O timer reinicia automaticamente após atingir o limite (RELOAD ou 0).
  - 0 (One-shot): O timer para após completar um ciclo (atingir RELOAD ou 0) e desabilita o bit TE.

### 4.2.4 Registradores PWM0CFG (0x1C) e PWM1CFG (0x20)

Estes registradores de 12 bits são a chave para a geração de formas de onda PWM complexas. Eles definem a ação da saída PWM em seis eventos distintos. Cada evento é controlado por 2 bits, com a seguinte codificação:

- 00: **No Action** (Mantém o estado atual da saída).
- 01: **Low** (Força a saída para nível lógico baixo).
- 10: **High** (Força a saída para nível lógico alto).

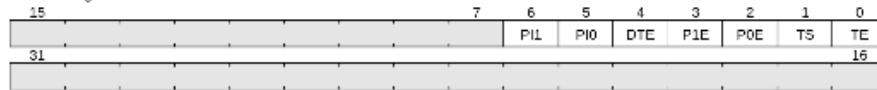
- **11: Invert** (Inverte o estado atual da saída).

Os campos de bits e seus eventos associados (verificados via RTL):

| Campo | Bits  | Evento Associado                                                                                                                    |
|-------|-------|-------------------------------------------------------------------------------------------------------------------------------------|
| E0    | 1:0   | Ação quando o Timer atinge <b>zero</b> ( <code>tmr_eq_zero</code> ).                                                                |
| E1    | 3:2   | Ação quando o Timer atinge <b>CMPX</b> durante a contagem <b>ascendente</b> ( <code>tmr_eq_cmpx</code> e <code>tmr_dir=1</code> ).  |
| E2    | 5:4   | Ação quando o Timer atinge <b>CMPY</b> durante a contagem <b>ascendente</b> ( <code>tmr_eq_cmpy</code> e <code>tmr_dir=1</code> ).  |
| E3    | 7:6   | Ação quando o Timer atinge <b>RELOAD</b> ( <code>tmr_eq_reload</code> ).                                                            |
| E4    | 9:8   | Ação quando o Timer atinge <b>CMPY</b> durante a contagem <b>descendente</b> ( <code>tmr_eq_cmpy</code> e <code>tmr_dir=0</code> ). |
| E5    | 11:10 | Ação quando o Timer atinge <b>CMPX</b> durante a contagem <b>descendente</b> ( <code>tmr_eq_cmpx</code> e <code>tmr_dir=0</code> ). |

**CTRL Register [Offset: 0x14, mode: w]**

Control Register.



| bit | field name | width | description                                                                                           |
|-----|------------|-------|-------------------------------------------------------------------------------------------------------|
| 0   | TE         | 1     | Timer enable                                                                                          |
| 1   | TS         | 1     | Timer re-start; used in the one-shot mode to restart the timer. Write 1 then 0 to re-start the timer. |
| 2   | P0E        | 1     | PWM 0 enable                                                                                          |
| 3   | P1E        | 1     | PWM 1 enable                                                                                          |
| 4   | DTE        | 1     | PWM deadtime enable                                                                                   |
| 5   | P10        | 1     | Invert PWM0 output.                                                                                   |
| 6   | P11        | 1     | Invert PWM1 output.                                                                                   |

**CFG Register [Offset: 0x18, mode: w]**

Configuration Register.



| bit | field name | width | description                                    |
|-----|------------|-------|------------------------------------------------|
| 0   | DIR        | 2     | Count direction; 10: Up, 01: Down, 11: Up/Down |
| 2   | P          | 1     | 1: Periodic, 0: One Shot                       |

Figura 4.2: Diagramas de Mapeamento de Bits para CTRL e CFG.



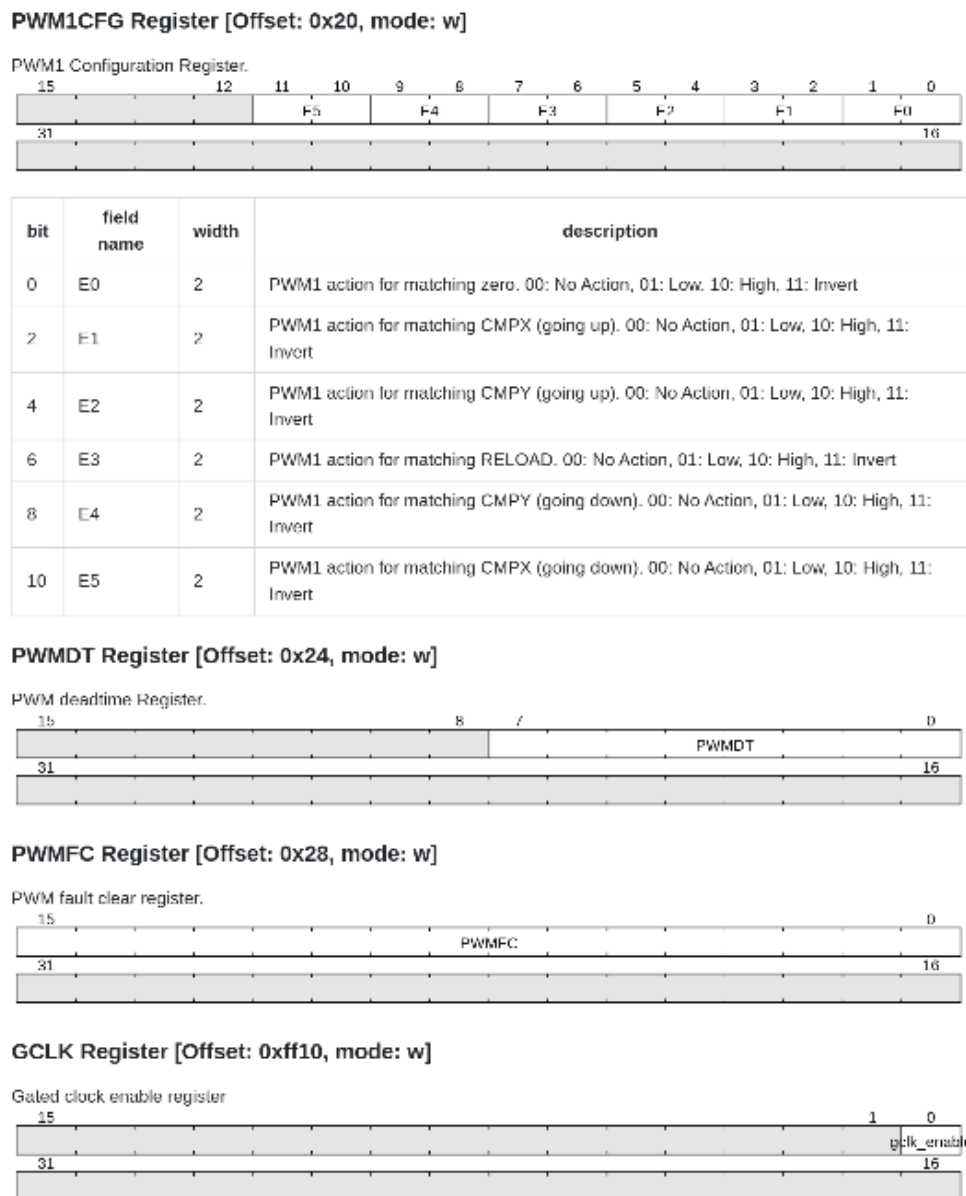


Figura 4.3: Diagramas de Mapeamento de Bits para PWM1CFG, PWMDT, PWMFC e GCLK.

#### 4.2.5 Registrador PWMDT (0x0024)

Este registrador de 8 bits (`pwm_dt`) define o período de *dead time* em ciclos de *tick* (saída do prescaler). Quando DTE (CTRL) está habilitado, as transições das saídas PWM são atrasadas por este valor, garantindo que um sinal se desative completamente antes que o outro se ative. O RTL revela que o `pwm1` atua como o complemento do `pwm0` com a banda morta inserida, o que é ideal para controle de meia-ponte.

### 4.2.6 Registrador PWMFC (0x0028)

Este registrador de 16 bits é usado para limpar a condição de falha do PWM. A limpeza não é um simples "escrever 1", mas uma **sequência de escrita específica**:

1. Escrever o valor 16'hA539 (PWM\_FAULT\_CLR\_C0) para ativar um sinal interno de pré-limpeza.
2. Em seguida, escrever o valor 16'hA953 (PWM\_FAULT\_CLR\_C1) para efetivamente limpar a flag de falha e reabilitar as saídas PWM.

A Figura 4.3 detalha seu mapeamento de bits.

### 4.2.7 Registradores de Interrupção (IM, RIS, MIS, IC)

O sistema de interrupções opera em três estágios, conforme verificado no RTL. A Tabela 4.3 detalham os flags de interrupção.

- **RIS (Raw Interrupt Status - 0xFF08) [R]**: Registrador de 3 bits que indica os eventos de interrupção brutos que ocorreram (Bit 0: Timeout, Bit 1: Match X, Bit 2: Match Y). É atualizado pelo hardware e é somente leitura.
- **IM (Interrupt Mask - 0xFF00) [RW]**: Registrador de 3 bits que mascara os eventos de interrupção. Um bit 1 permite que o evento correspondente no RIS contribua para o sinal IRQ; 0 o mascara.
- **MIS (Masked Interrupt Status - 0xFF04) [R]**: Registrador de 3 bits que reflete o status das interrupções após a aplicação da máscara ( $MIS = RIS \text{ AND } IM$ ). Se qualquer bit aqui for 1, o pino IRQ externo é ativado.
- **IC (Interrupt Clear - 0xFF0C) [W]**: Registrador de 3 bits para limpar as interrupções. Escrever 1 em um bit limpa o bit correspondente no RIS. O registrador é auto-limpante após a escrita.

Tabela 4.3: Flags de Interrupção [1].

| Bit | Flag | Width | Descrição                                                       |
|-----|------|-------|-----------------------------------------------------------------|
| 0   | TO   | 1     | Timeout: TMR matches 0 (down counting) or RELOAD (up counting). |
| 1   | MX   | 1     | TMR matches CMPX register.                                      |
| 2   | MY   | 1     | TMR matches CMPY register.                                      |

#### 4.2.8 Registrador GCLK (0xFF10)

Este registrador de 1 bit controla o *Clock Gating* do núcleo do timer. A Figura 4.3 detalha seu mapeamento de bits.

- **Bit 0 (gclk\_enable):** 1: Habilita o clock para o núcleo do timer. 0: Desabilita o clock para o núcleo, reduzindo o consumo de energia. O acesso aos registradores via Wishbone permanece funcional.

## Referências

- [1] Efabless. *EF\_TMR32 GitHub Repository*. Disponível em: [https://github.com/efabless/EF\\_TMR32](https://github.com/efabless/EF_TMR32)
- [2] Universidade Federal de Campina Grande - VIRTUS CC. *Wishbone RISC-X*.
- [3] OpenCores. *Wishbone Bus Architecture Specification B4*.